GRADUATION PROJECT

Project documentation

Multi-Agent Refactoring for Python

The specification and technical plan for the first release. Prepared for the project team and supervisor, with implemented behavior distinguished from planned work.

Team
Abdelrahman Magdy
Abdullah Tarek
Abdelrahman Adel
Shahd Abdelhay
Dania El-Walily

Scope: Python; fine-tuning an existing model; no paid-services budget.

Schedule: an initial three-week delivery window from the agreed kickoff. Official dates remain to be confirmed.

Status: foundation implemented; model training, final evaluation, and the complete product remain planned work.

Private GitHub repository and editable sources

Version 0.3. Read task status in GitHub Issues and the Project board.

Contents

Project proposal 3
Software requirements 5
Three-week delivery plan 7
Team and workstreams 9
System design 10
Technical decision log 12
First-day guide 13
Development environment 14
Working together 15
Testing and verification 16
Related-work review plan 17
Data preparation plan 18
Fine-tuning plan 19
Model card 20
Evaluation protocol 21
Project risks 23
GitHub workspace 24
Supervisor reviews 25
Supervisor review record 26
Foundation user guide 27
Initial delivery checklist 28
Optional single-laptop agent execution pack 29
Methodology and tool references 30

Project proposal

Source: docs/proposal/project-proposal.md

Working title: Multi-Agent Refactoring for Python **Status:** planning draft for supervisor review **Initial delivery window:** three weeks from the agreed team start date

Problem

Refactoring requires more than producing code that looks cleaner. A useful change must address a specific structural problem and preserve the program's intended behavior. Language models can propose changes that are unnecessary, invalid, or behaviorally incorrect. This project investigates whether separating analysis, modification, and review into coordinated roles improves the reliability of a small refactoring workflow.

The intended user is a student or developer reviewing a small Python function or file. The system will present a proposed change and the evidence available to assess it. It will not modify the user's original files automatically.

Proposed approach

An analysis agent identifies a supported refactoring opportunity. A refactoring agent proposes a change. A review agent examines the change alongside deterministic checks. The interface presents the original and proposed code, a diff, an explanation, and separate verification results.

The roles may share one underlying code model. The team will fine-tune an existing open-weight model for a narrowly defined refactoring task, initially targeting the modification role. The original model will remain available as a baseline. Expanding fine-tuning to other roles depends on the feasibility results and available time.

Objectives

- Deliver a reproducible, locally runnable workflow for selected Python refactorings.
- Prepare a documented dataset with source-level separation between training, validation, and test sets.
- Run a traceable fine-tuning experiment and retain the adapter and configuration.
- Measure the effects of fine-tuning and agent coordination separately.
- Report failure cases, resource use, and limitations as well as successful examples.

First-release scope

The first release will accept a function or a small Python file. One refactoring type will be selected after the feasibility study, with reference examples and acceptance rules for each. The interface will support input, inspection of the proposed change, verification results, and explicit user acceptance or rejection.

Paid model APIs and paid hosting are outside the budget. Public deployment is not required for the initial delivery. Test execution, if enabled, will be restricted to trusted examples in a separately reviewed runner.

Training a language model from scratch, supporting multiple languages or large repositories, automatically changing GitHub branches, and guaranteeing behavioral equivalence for all programs are outside this release.

Research questions

1. Does a coordinated analysis, modification, and review workflow produce more acceptable refactorings than a single-agent workflow on the selected tasks? 2. How does task-specific fine-tuning change performance relative to the original model? 3. What quality improvements, if any, justify the additional inference time and resource use?

The proposed contribution is an empirical study of this bounded workflow. Improvement is a hypothesis, not a promised result. Dataset preparation, reproducibility, and failure analysis are part of the contribution. The supervisor will confirm whether the proposed scope meets the department's academic requirements.

Deliverables and success criteria

Deliverables are a working end-to-end prototype, documented data preparation, a recorded fine-tuning experiment, an evaluation report, developer and user documentation, and a presentation with a repeatable demonstration.

Prototype completion is assessed against the agreed acceptance criteria. Research findings are assessed through the evaluation protocol, including negative results. Numerical performance targets will be recorded after the baseline study and before the final test set is evaluated.

Resources and supervision

The team has five members with similar reported experience. The repository owner coordinates the work. Hardware and weekly availability will be recorded at kickoff. Each phase ends with a supervisor review, written decisions, and follow-up Issues.

Decisions for the first supervisor review

Confirm the problem and proposed contribution; select the refactoring types; review the training resources; confirm the university report template and official dates; and agree on the evaluation approach.

Software requirements

Source: docs/requirements/srs.md

This is the first-release specification. The current implementation provides static analysis only. P0 requirements are essential to the agreed release; P1 requirements are included only if time permits.

ID	Requirement	Acceptance criterion	Priority
FR01	Accept bounded Python input	Accept non-blank text up to 20,000 characters; reject larger or malformed requests with an understandable response	P0
FR02	Analyze code	Return structured findings with locations and reasons, or an empty result; the foundation rules cover long functions, including async functions, and nested control blocks	P0
FR03	Propose a refactoring	Return proposed code, a reason, and the supported refactoring type, or an explicit failure	P0
FR04	Verify a proposal	Present syntax checks and available test results separately from the agent's review	P0
FR05	Show a diff	Display added and removed lines and the affected components	P0
FR06	Preserve user control	Allow acceptance, rejection, and download without automatically modifying the original input	P0
FR07	Trace each run	Record an identifier, configuration, model version, state, duration, and failures	P0
FR08	Bound execution	Enforce declared limits on calls, retries, generated tokens, and elapsed time	P0
FR09	Compare experiments	Evaluate the agreed variants on the same test cases and export results	P0
FR10	Retain run history	Retrieve previous results on the demonstration machine	P1

Quality requirements

- NFR01: The API must not execute submitted or generated code. The foundation uses Python AST parsing only.
- NFR02: Any later test execution must use a separate runner with CPU, memory, time, network, filesystem, and privilege limits. The initial release uses trusted examples.
- NFR03: Basic operation must not require a paid API or committed credentials.
- NFR04: The core environment uses Python 3.11 and a lockfile. GPU training dependencies are resolved separately for the actual hardware.
- NFR05: Experiment records must identify the code, data, model revision, configuration, seed, and hardware.
- NFR06: Passing tests must not be presented as proof of full behavioral equivalence.
- NFR07: Response-time and memory targets will be set after measurements on the demonstration machine during P1.

Main use cases

UC01: Submit a valid example, inspect the proposed change, and download an accepted result.

UC02: Submit invalid Python and receive an error without executing it.

UC03: Encounter a generation or verification failure and see a clear reason rather than a success indicator.

UC04: Run a comparison experiment and export case-level records and a summary.

Traceability

Implementation Issues identify the relevant requirement IDs. Pull requests include acceptance evidence. Delivery includes a matrix of implemented, deferred, and excluded requirements with links to tests. Requirements that affect the academic scope are confirmed through a recorded supervisor review.

Three-week delivery plan

Source: docs/planning/roadmap.md

The first delivery is planned for 21 calendar days from the team's agreed kickoff. Calendar dates and the official university deadline remain unconfirmed. This replaces the previous schedule. Repository setup is complete; the product phases are planned work, not completed milestones.

Phase	Window	Required outcome	Supervisor review
P1 Scope and feasibility	Days 1-2 (Week 1)	Hardware inventory, one supported refactoring type, measured local inference, tiny LoRA save/reload smoke test or a recorded blocker	R1 on Day 2: scope and resource decision
P2 Data and baseline	Days 3-5 (Week 1)	Small licensed dataset, source-separated splits, base-model baseline, shared API/role contracts, frozen comparison protocol	R2 on Day 5: data, baseline and contracts
P3 Integration and first training run	Days 6-10 (Weeks 1-2)	One real-model input-to-result case, initial frontend, adapter training and validation evidence if resources permit	R3 on Day 10: demo, actual training status and failures
P4 First-release integration	Days 11-15 (Weeks 2-3)	Three bounded roles, trusted-example verification path, honest status display, acceptance checks and feature freeze	R4 on Day 15: readiness for final evaluation
P5 Evaluation and initial delivery	Days 16-21 (Week 3)	Frozen-test comparison, critical fixes, clean setup, report, presentation, demo and release	R5 by Day 21: delivery and next priorities

Scope that fits the window

Start with one Python file/function and one refactoring type selected using real examples on Day 2. One local web interface, one FastAPI process, one local model shared by three logical roles, and at most one correction attempt are sufficient. Use local files or SQLite for run records. Authentication, public hosting, repository-wide edits, extra languages, multiple model services and optional orchestration frameworks are deferred.

Fine-tuning an existing open-weight model remains a research objective. The plan cannot guarantee its completion before checking hardware. By Day 2, measure inference and a tiny adapter run, including save and reload. If the laptop cannot support training, reduce model/context/batch size or obtain confirmed free university GPU access. Record the blocker and discuss it at R1. A base-model demonstration is useful but does not satisfy the fine-tuning deliverable. Never report it as trained.

Critical path and daily rhythm

Days 1-2: resources and scope. Days 3-5: data/splits, baseline and contracts. Days 6-10: provider, roles, API and UI converge on a real-model demonstration. Days 11-15: verification and model selection. Days 16-21: final evaluation and delivery.

Use a short daily update in Issues: completed output, next action, blocker and evidence link. Keep one primary implementation task active per person. Reserve Day 21 for final checks and the supervisor discussion, not new features. A phase ending does not automatically complete its tasks.

Schedule the five supervisor discussions at kickoff. Record real feedback after each phase; never mark a review approved before it happens. Independent work can continue while feedback is pending. Scope, data-split and evaluation changes awaiting a decision remain pending.

Resource and delivery gates

Freeze the test split by Day 5 and never use it for tuning. Demonstrate a real model by Day 10. Freeze core features and model choice by Day 15. Use Days 16-20 for test evaluation, failure analysis, critical fixes and report assembly. If a gate is missed, reduce optional scope and explicitly record incomplete requirements with the supervisor; do not silently change acceptance criteria.

A clean clone and new environment on the same laptop proves clean setup only. Record whether a second physical machine was actually tested. The optional agent execution pack follows the same gates and evidence requirements.

Ownership and follow-up

See the person-by-week matrix and tasks grouped by owner. Each task has one accountable owner and no assigned peer reviewer. Each member supplies notes for their work; Dania assembles delivery materials. The optional agent execution pack does not change human ownership. Improvements after Day 21 require a separate plan based on evaluation and supervisor feedback.

Team and workstreams

Source: docs/planning/team.md

Member	GitHub	Main workstream
Abdelrahman Magdy	abdulrhmanmagdy1	Backend, integration, and coordination
Abdullah Tarek	abdallah414	Agent workflow and verification
Abdelrahman Adel	abdelrhmanadel200	Model selection, fine-tuning, and inference
Shahd Abdelhay	shahd-abdelhay	Data, research, and evaluation
Dania El-Walily	d5ania	Frontend, report assembly, and presentation

The team delivery plan states each member's outcomes in all five phases. Each Issue has one accountable owner. There is no assigned peer-review role. Owners run checks and publish evidence before completing work; supervisor reviews remain part of every phase.

Magdy coordinates priorities, component handoffs, and supervisor meetings. Documentation is shared: members write the technical content for their own work, and Dania assembles the report and presentation. Model training and agent implementation are separate workstreams led by Adel and Abdullah, with an explicit interface between them.

All four teammates have accepted repository access; native task assignments use their GitHub accounts. Email addresses are not published here.

System design

Source: docs/architecture/system-design.md

The proposed design separates the interface, workflow coordination, model inference, and verification. The current implementation is a small FastAPI service for static analysis. The product interface, model provider, agents, persistent run history, and test runner are not yet implemented.

System context

Proposed design; not yet implemented.



Proposed components

Proposed design; not yet implemented.



Roles exchange versioned JSON schemas. Proposed run states are queued, analyzing, generating, verifying, completed, and failed. Results keep syntax validity, test availability, test outcomes, and the agent's verdict separate. Input code is data; it does not grant the model permission to execute commands or change the host.

Runtime sequence

Validate input size and structure, collect static indicators, and run bounded analysis, generation, and review. Record the run identifier, settings, model revision, timing, and failures. Present the change for explicit user review before acceptance.

A failed check keeps the proposal rejected or uncertain. The initial experiment allows at most one correction attempt; changing this limit is an experimental decision that must be recorded.

Implemented API

- GET /health returns service health, version, and scaffold mode.

- POST /api/v1/analyze accepts a code field and returns syntax_valid, findings, and error. The engine is static heuristics, not a language model.

- GET /docs and /openapi.json provide the generated API reference.

Planned contracts

POST /runs, GET /runs/{id}, and acceptance actions will be specified before interface implementation. They are not available endpoints yet. Begin with an explicit workflow; adopt LangGraph only if a small comparison shows a practical benefit for state handling or recovery.

Storage and deployment

The foundation needs no external database. SQLite is a candidate for local run history; PostgreSQL is justified only if concurrent or multi-user access requires it. React and TypeScript are candidates for the product interface after the contracts are agreed. The FastAPI reference page is a development aid, not the final interface.

The first release is local. Microservices and Kubernetes are outside the current scope.

Trust boundaries

The analysis service must not read arbitrary host files or execute input. Any test runner must be separate, non-root, resource-limited, without secrets, network access, host paths, or the Docker socket. Docker alone is not sufficient grounds to expose arbitrary code execution to the public. The initial release remains limited to trusted examples.

Technical decision log

Source: docs/architecture/decisions.md

Record context, alternatives, consequences, and evidence for significant decisions. Python input, fine-tuning, GitHub collaboration, and a zero paid-services budget were agreed by the team owner. Other choices remain subject to feasibility and supervisor review.

ID	Decision	Status	Rationale
ADR01	Python as the input language	Agreed	Limits the task and simplifies verification
ADR02	Fine-tune an existing model	Agreed	Avoids training a code language model from scratch
ADR03	GitHub for shared code, documentation, and tasks	Agreed	Keeps changes and review together
ADR04	Local operation and no paid services	Agreed	Matches the budget
ADR05	Private repository owned by abdulrhmanmagdy1	Implemented	Preserves the requested ownership and access
ADR06	Python 3.11, uv, and FastAPI	Foundation choice	Provides a locked core environment and inspectable API
ADR07	Fine-tune the modification role first	Proposed	Keeps the first training task bounded
ADR08	Small open-weight model with LoRA	Proposed	Model size and precision depend on the measured hardware limits
ADR09	SQLite for local run history	Proposed	Avoids an external database before one is needed
ADR10	Explicit workflow before a dedicated agent framework	Proposed	Establishes behavior before adding framework complexity
ADR11	GitHub storage for distributable artifacts	Feasibility check	Confirm size, quota, and licensing before uploading weights or data

New decision record

Include a title, status, date, owner, context, alternatives, decision, consequences, evidence, affected requirements, and a condition for revisiting the choice.

First-day guide

Source: docs/development/first-day.md

Accept the repository invitation, then open the Project board and documentation index.

1. Read the proposal and roadmap. Raise scope concerns in an Issue before changing the plan. 2. Follow the environment setup guide. Add a short doctor and test result to Issue #2 without credentials or personal information. 3. Record hardware and weekly availability in Issue #1. 4. Read your workstream and phase outcomes in the team delivery plan. Start one primary task after its dependencies are ready, and tell the coordinator if availability or hardware requires a change. 5. Submit work in a linked pull request, include verification evidence, and run the acceptance checks before merging.

Use assignee:@me on the Project board to find assigned work, or label:phase:P1 to focus on the first phase. Issues contain task details and dependencies; milestones group phase outcomes. Project access is separate from repository access.

Do not begin a large training run before agreeing on hardware, data, and the baseline. The foundation service currently performs static analysis only.

Development environment

Source: docs/development/setup.md

The core environment runs on CPU and does not download a model. Training uses a separate environment matched to the available GPU. Keep credentials and local data outside Git.

Requirements

Install Git, Python 3.11, and uv from their official sources. Repository access is required. Node.js will be needed when the product interface is implemented; there is no frontend application in this foundation.

Setup

```
git clone https://github.com/abdulrhmanmagdy1/grad-refactor-ai.git
cd grad-refactor-ai
uv sync --frozen
uv run python scripts/doctor.py
uv run pytest
uv run uvicorn refactor_ai.api:app --reload --host 127.0.0.1
```

These uv commands also work in Windows PowerShell without activating the virtual environment manually. Open http://127.0.0.1:8000/docs and try POST /api/v1/analyze. The service parses code but does not execute it.

Before a pull request

```
uv run ruff check .
uv run ruff format --check .
uv run pytest
uv run python scripts/check_docs.py
```

Training environment

Follow the training plan after confirming the GPU and VRAM. Select a compatible PyTorch and CUDA build for the actual machine. Do not install a CUDA configuration on a Mac or CPU-only machine by assumption.

training/requirements.in lists candidate dependencies. Pin a separate training environment after the GPU smoke test. The core uv.lock does not reproduce the GPU training environment.

Local files

data/, models/, runs/, .env, and .venv/ are excluded from Git. Commit sanitized experiment records under experiments/. Distributable artifacts may use GitHub Releases after size and licensing checks. Record the base-model identifier and revision rather than committing another copy of its weights.

Troubleshooting

For access errors, check the account and invitation status. For Python version errors, install 3.11 or use uv's Python management. For training memory errors, reduce context, batch size, or model size and change one factor at a time. Keep paid services outside the agreed plan.

Working together

Source: CONTRIBUTING.md

Start from the assigned Issue and confirm its dependencies and acceptance criteria. Each member owns a main workstream and normally keeps one primary implementation task active.

Publish work

```
git switch main
git pull --ff-only
git switch -c feat/123-short-description
```

Keep the change focused, run the relevant checks, and update its documentation. Open a linked pull request using Closes #123 so the team can see the output and automated checks. A second-person approval is not required; the task owner checks the result and merges it when ready. Do not force-push main.

Research and meeting tasks can be completed with a linked record instead of code. Uploading a file alone is not completion: the result must meet the stated acceptance criteria. Training work includes versions, settings, artifacts, and actual results.

Status

Backlog is planned work. Ready means dependencies allow it to start. In Progress means work has begun. In Review is optional for a result awaiting a decision or supervisor feedback; it does not create a peer-review assignment. Done means the owner has completed the acceptance checks and published the output.

Record blockers and the help needed in the Issue. The Project helps update status automatically, but owners keep it accurate. Every phase ends with a supervisor review and recorded decisions.

Data and execution

Do not commit credentials, private contact details, unlicensed data, or large model weights. Record small sanitized experiment logs and checksums. The API must not execute submitted code. Enable a separate runner only after completing its isolation checks on trusted examples.

Testing and verification

Source: docs/development/testing.md

Foundation checks

The tests cover health reporting, blank and oversized input, invalid syntax, long async functions, and structural nesting. A submitted example containing a file-write operation verifies that analysis does not execute that example. Dataset-validator tests cover empty or missing splits, source-group overlap, and normalized duplicates.

These tests verify the foundation. They do not establish model quality, complete refactoring behavior, or runner isolation.

First-release test plan

- Unit tests for role schemas, transformations, and tools.
- Integration tests for run transitions, model failures, timeouts, and bounded retries.
- Acceptance tests traced to the agreed requirements.
- Regression examples with checked reference tests.
- Runner checks for resource limits, isolation, process cleanup, and absence of network, secrets, and host access.
- A clean installation and full demonstration in a fresh environment before delivery, preferably on a second physical machine; record the actual hardware coverage.

Tests generated by the same model are not sufficient evidence for its own changes. Combine reference tests with a declared assessment rubric. Passing tests does not prove complete semantic equivalence.

Continuous integration

GitHub Actions runs lint, format, pytest, and local Markdown-link checks on Linux and Windows using the locked core environment. It does not train models, download weights, or execute user-submitted code. Monitor the free Actions allowance; paid overages are not part of the plan.

Related-work review plan

Source: docs/research/related-work.md

The literature review has not been completed. Its purpose is to identify a defensible research question and comparisons that can be reproduced with the team's resources.

Search areas

Review primary work on language-model-based refactoring, multi-agent code improvement, parameter-efficient fine-tuning for code, code smells, and behavior-preservation evaluation. Distinguish refactoring from bug fixing and deliberate functional changes.

Reading record

For each paper, record its authors, year, link, problem, language, data and license, model and hardware, roles and tools, baseline, metrics, findings, limitations, and reproducibility requirements. Prefer a focused set of directly relevant studies over a long unexamined bibliography.

Decisions supported by the review

Use the review to choose task types, data sources, a practical baseline, and the evaluation method. Do not claim novelty or superiority without evidence. Discuss the proposed gap during P1 and the comparison method during P2.

Data preparation plan

Source: docs/data-and-training/data-plan.md

Record format

Each record contains id, source_group, split, prompt, completion, license, and refactoring_type. The prompt includes the task and original code. The completion is a reviewed reference output. A separate manifest records source URLs, commits, and reference tests.

Collection and curation

Use sources with terms compatible with the intended training and distribution. Record the license for each source; public availability on GitHub is not a substitute for a license review.

Check Python syntax before and after each change. Exclude functional changes, ambiguous examples, credentials, and personal data. Manually prepared teaching examples or controlled transformations are possible if reviewed. Correctness and source diversity matter more than an untested volume target.

Splitting and leakage checks

Begin with a candidate 80/10/10 train/validation/test split by repository or example family, not by randomly distributing rows. Review the ratio against the available sample size. A source_group must not cross splits.

Check duplicate prompts and targets and investigate near-duplicates. The current validation script detects normalized exact duplicates and overlapping groups; it does not detect semantic similarity. A stronger near-duplicate check remains a data task.

Use validation data for selection. Freeze test data before final model and prompt choices. Save dataset hashes and split manifests. Do not reuse test examples as prompt demonstrations while tuning.

Data acceptance

Require clear provenance and licensing, a recorded sample review, complete fields, valid reference code, source-separated splits, a curation log, and reference tests for behavior-preservation measurements. Final dataset size depends on the sources and feasibility results.

GitHub storage

Small permitted examples and manifests may be committed. Larger permitted data requires a Releases or LFS quota check. If the free allowance is insufficient, reduce the dataset or agree on reproducible source-download scripts. No paid storage or additional external storage service has been approved.

Fine-tuning plan

Source: docs/data-and-training/training-plan.md

No model, hardware configuration, or training result has been approved yet. The first candidate task is the refactoring role, while the original model remains the comparison baseline.

Feasibility experiment

Inventory hardware, select small candidates with suitable licenses, measure inference on the demonstration machine, and run a small training job. Check memory, elapsed time, adapter saving, reloading, and checkpoint recovery.

Qwen Coder is one example candidate, not a final selection or a claim of best performance. Record exact model and tokenizer revisions after selection.

Candidate training path

The preliminary path uses PyTorch, Transformers, Datasets, and PEFT. Begin with LoRA if the hardware supports it. Consider quantization only after a compatibility test. The current script implements an unquantized CUDA LoRA path; QLoRA is not implemented.

Do not assume that CPU training or an intermittently available free GPU can meet the schedule. Set the model, precision, context length, batch settings, and training budget from measurements.

Preparation and execution

Validate the curated JSONL file:

```
uv run python scripts/validate_dataset.py data/curated.jsonl
```

In a separate, hardware-compatible training environment, copy the configuration example, select an approved model and exact revision, and run:

```
python training/train_lora.py --config configs/training.local.json
```

The script rejects unresolved settings, invalid data, and environments without CUDA. It masks prompt tokens from the loss and excludes test records from the Trainer. It supports --resume-from-checkpoint. A GPU smoke test is required before relying on it for experiments.

Experiment record

Record the run identifier, code SHA, dataset hash and split manifest, model revision, package and CUDA versions, hardware, seed, LoRA settings, batch and accumulation settings, learning rate, epochs, context length, duration, peak memory, validation results, adapter location and checksum, and any failure or interruption. Use experiments/run-template.json.

Selection and reporting

Choose a candidate using declared validation metrics, then evaluate the agreed test set. Training loss alone does not measure successful refactoring. If fine-tuning does not improve results, investigate and report that outcome without tuning on the test set.

Free resources and storage

Colab may support interactive training experiments within its limits, but it is neither guaranteed compute nor permanent hosting. Save resumable artifacts within the available quota. Prefer a small adapter and metadata over redistributing the base model.

Model card

Source: docs/data-and-training/model-card.md

Status: not trained. Complete this card from the first actual experiment; it is not evidence that a trained model exists.

Field	Current value
Intended task	Selected refactorings for small Python inputs
Base model and revision	Not selected
License and adapter terms	Not reviewed
Training data and provenance	Not prepared
Adaptation method	LoRA is the initial candidate
Hardware and training duration	Not measured
Test set	Not approved
Results and baselines	No results yet
Artifact and checksum	Not available

The intended use requires human review and does not guarantee behavioral preservation. Add measured context limits, supported task types, and observed failure cases after evaluation.

Evaluation protocol

Source: docs/evaluation/protocol.md

This draft is to be agreed during P2 before the final test evaluation. The comparison separates the effect of fine-tuning from the effect of role coordination.

Comparison groups

Group	Model configuration	Workflow
A	Original model	One role with the same available verification tools
B	Original model	Analysis, refactoring, and review roles
C	Fine-tuned modification model	One role
D	Fine-tuned modification role; original model for other roles	Analysis, refactoring, and review roles

Hold data, tool versions, task definitions, and generation settings constant where practical. Declare a total call and token budget and explain its distribution across roles. If equal resource budgets are impractical, report quality alongside elapsed time, calls, and tokens rather than attributing every difference to coordination.

Measures

- Syntax validity: the proportion of outputs that parse as Python.

- Reference-test preservation: the proportion passing reference tests among cases with those tests, with coverage and case counts shown.

- Refactoring success: compliance with a task-specific rubric without an unintended functional change apparent in the available evidence.

- Overall acceptance: the declared combination of syntax, available reference tests, and rubric, with coverage limitations stated.

- End-to-end and stage latency, calls, tokens, and memory where measurable.

- Task-appropriate structural indicators, such as function length or complexity. A lower indicator alone is not sufficient evidence of success.

Include failures and timeouts in the denominator. Compare the same examples across groups. Report sample size and source diversity, and repeat seeds if resources allow. Do not imply statistical confidence unsupported by the sample.

Qualitative assessment

Shahd assesses a sample using the fixed task rubric and records ambiguous cases and failure patterns. This is evaluation of model output, not a peer-review assignment between teammates. Model-generated examples are not automatic ground truth.

Case-level record

Record case ID, source group, split, data/model/code versions, group, seed, input and output references or hashes, syntax and test results, rubric assessment, timing, calls, tokens, failure type, and evaluator. Preserve the full test result record rather than a selected set of successes.

Release acceptance

The agreed P0 behavior must run on the demonstration machine and from a clean clone in a fresh environment. Prefer a second physical machine; if unavailable, label the reproduction as same-laptop clean setup and record cross-machine validation as untested. The release needs reproducible results, an error analysis, documented limitations, and a supervisor review. Numerical targets are set after the baseline and before the final test evaluation. A working prototype does not depend on proving that the research hypothesis is true.

Project risks

Source: docs/planning/risks.md

Risk	Early signal	Response	Responsible role
No suitable GPU	Training smoke test fails in P1	Reduce model, context, or task; discuss university resources	Coordinator and training owner
Interrupted free compute	Sessions terminate	Record checkpoints and recovery; keep demonstration hosting independent	Training owner
Weak data	Invalid changes or unclear licenses	Review samples, reference tests, and source suitability	Data owner
Data leakage	Related projects or examples cross splits	Split by source and check near-duplicates	Evaluation owner
Fine-tuning regression	Validation is worse than baseline	Review data and settings; retain and report the baseline	Training owner
Late integration	No full case by Day 10	Prioritize one complete path and defer extras	Integration owner
Scope growth	More languages or roles without evidence	Enforce the agreed release scope	Team
Unsafe execution	Untrusted code reaches execution	Keep execution disabled until runner review; use trusted MVP examples	API and verification owners
Delayed supervisor review	No appointment before phase end	Arrange early; prepare written evidence; continue independent work	Coordinator
GitHub quota limits	Artifacts exceed the free allowance	Check early; prefer small adapters and manifests	Coordinator
Incomplete team access	A member cannot open the repo or board	Verify both permissions and invitation acceptance	Repository owner

GitHub workspace

Source: docs/planning/github-workspace.md

The private repository is owned by abdulrhmanmagdy1. grad-refactor-ai is a working repository name.

- Repository
- Project board
- Tasks and Owners view
- Issues
- Milestones
- Assignment table
- PDF editions

Where work belongs

Issues contain tasks, evidence, and dependencies. Milestones group phase outcomes. The Project provides status and ownership views. Source documentation lives in docs/ rather than a separate Wiki. Pull requests record changes and check results, and Actions run the foundation checks. Releases may hold permitted artifacts after quota and license checks.

Board states

Backlog contains planned work that has not entered execution, including tasks waiting on earlier outcomes. Ready means the task is defined and its dependencies allow it to start. In Progress means work has started. In Review is optional when an output is waiting for a decision or supervisor feedback. Done means the output has been accepted. Use blocked as a label with a written reason and dependency link.

Team Board shows status columns. Tasks and Owners groups a table by milestone. The Project automatically adds matching new Issues, changes linked work to In Progress, and marks closed work Done. A draft pull request may require moving the task back to In Progress manually. Automation does not verify acceptance criteria.

Assignments and access

There are 33 planned tasks plus the completed setup Issue #34. Each task has one accountable owner. The delivery plan defines stable workstream ownership and outcomes across all five phases; no peer reviewer is assigned.

All four teammates have accepted repository access and have Project write access. Every planned Issue has a native assignee. Current access is determined by GitHub settings, not this snapshot.

Notifications

Each member chooses repository notification preferences and follows assigned Issues and review requests. Uploading a commit alone is not a reliable team completion signal. Open a linked pull request, include the check results, and update the Issue.

Main-branch policy

The current GitHub plan rejected enforced branch protection for this private repository. The project remains private and has no paid upgrade. Owners complete acceptance checks before merging; a second-person approval is not part of the team workflow. Squash merging and deletion of merged branches are enabled.

By Owner: live Project view groups all tracked work by its native GitHub assignee.

Supervisor reviews

Source: docs/supervisor-reviews/README.md

Hold a review after P1, P2, P3, P4, and P5. Each review has an Issue linked to its milestone. No appointment or academic approval is implied by the existence of a task.

Before the meeting

Prepare a short summary, links to outputs, a demonstration where available, actual results, open problems, alternatives, and specific decisions needed. Arrange the review before the phase ends. No supervisor messages or calendar invitations are sent automatically.

After the meeting

Copy the review template, record the date and attendees, identify the exact version shown, and record feedback, agreed decisions, and pending questions. Create linked follow-up Issues. Distinguish a team proposal from a supervisor decision. Continue independent work while waiting; hold decisions that change scope or evaluation until agreement.

Supervisor review record

Source: docs/supervisor-reviews/review-template.md

Status: blank template; this does not represent a meeting that has taken place.

- Phase, date, and attendees:
- Release or code SHA and links to the demonstration and results:
- Completed and incomplete outcomes:
- Problems and alternatives:
- Questions requiring a decision:
- Supervisor feedback:
- Agreed and pending decisions:
- Follow-up Issues, owners:
- Conditions for moving to the next phase:

Foundation user guide

Source: docs/delivery/user-guide.md

The current service supports environment verification and static analysis. It does not yet provide a refactoring product.

Follow the setup guide, open http://127.0.0.1:8000/docs, select POST /api/v1/analyze, and choose Try it out. Submit Python text in the code field. Inspect syntax_valid, findings, and error.

No model or API key is needed. Findings are limited structural heuristics and do not cover every code-quality issue. The foundation does not modify code, download a refactoring, run user tests, or start training.

The planned interface will support code input, a diff, an explanation, verification results, and user acceptance or rejection. Update this guide when that behavior is implemented and verified.

Initial delivery checklist

Source: docs/delivery/checklist.md

- Confirm the university template and official dates with the supervisor.
- Trace implemented, deferred, and excluded requirements to evidence.
- Reproduce setup and a complete run from a clean clone and fresh environment; prefer a second machine and state actual hardware coverage.
- Record data licensing, curation, independent splits, and versions.
- Record fine-tuning settings and a reloadable adapter within storage and license limits.
- Compare original and adapted models, and single-role and coordinated workflows.
- Include failures, resource use, and limitations in the evaluation.
- Update developer and user guides and the model card.
- Write the academic report using the university template: problem, related work, method, design, implementation, experiments, results, limitations, and future work.
- Prepare the presentation, repeatable demonstration, and a small backup recording.
- Tag the release and publish notes, checksums, and experiment links on GitHub.
- Record the P5 review and prioritize subsequent development.

The final report, presentation, and demonstration recording are not complete at the foundation stage. They must reflect the work and results actually produced.

Optional single-laptop agent execution pack

Source: docs/agent-execution/README.md

This folder is a contingency implementation brief for Codex or Claude working in this repository on one laptop. It prepares the work; it does not claim the implementation is complete. The five students retain the workstream ownership in the team plan, even when one coding agent implements several components. Humans inspect the final system and explain its work to the supervisor.

Start here

1. Clone the private repository using the laptop owner's normal GitHub authentication. Never paste credentials into a prompt or commit them. 2. Open the repository root in the coding tool. Read root AGENTS.md and this folder in the order below. 3. Paste START-PROMPT.md. Start with repository inspection and the hardware gate, not a new scaffold. 4. Work through the ordered steps, keeping actual evidence and checkpoints. Resume from state, not memory. 5. Deliver the final handover for team inspection. Supervisor meetings and human feedback are recorded only when they occur.

Reading order

File	Purpose
START-PROMPT.md	Copyable instruction to start or resume the implementation
01-project-brief.md	Goal, first-release scope, existing code, exclusions and final outcome
02-execution-plan.md	Ordered steps, file destinations, dependencies, commands and exit evidence
03-component-contracts.md	Proposed interfaces, data records, state transitions and failure behavior
04-data-model-evaluation.md	Data preparation, hardware decisions, training and independent evaluation
05-working-and-recovery.md	Single-laptop setup, Git workflow, state, restart and blocker handling
06-acceptance-and-handover.md	Completion checklist and the team's final inspection
execution-state.template.json	Machine-readable checkpoint template to copy before work begins

Source of truth

Use the actual repository and tests for implementation status. Use requirements, architecture, data plan, training plan and evaluation protocol for the full specification. Use planning/backlog.json and planning/assignments.json for task dependencies and owners. This folder proposes concrete defaults where a decision is still open; log changes in the architecture decisions before implementation.

Three weeks means Days 1-21 from agreed kickoff, with phase gates on Days 2, 5, 10, 15 and 21. Agent execution does not remove hardware limits, genuine research work, or the supervisor discussions. Do not invent datasets, training runs, successful tests, approvals or citations.

Methodology and tool references

Source: docs/references.md

These references support project organization and candidate tooling. The related-work review is a separate research task.

- arc42 overview: architecture goals, constraints, decisions, and risks.
- C4 diagrams: system context and component views.
- Diataxis: explanation, tutorials, procedures, and reference material.
- GitHub planning: Issues and Projects.
- Protected branches: review controls and plan availability.
- Large files on GitHub: Git storage limits.
- FastAPI: the API and generated reference.
- uv installation: the core environment.
- PEFT LoRA: parameter-efficient adaptation.
- Colab FAQ: free-compute limitations.
- LangGraph: a candidate for workflow state and coordination.